

Math Templates

Quick Reference Table

#	Name	Description	Algorithm	Variation from Template	Time	Space
7	Reverse Integer	Reverse the digits of a signed 32-bit integer	Pop digits via <code>%10</code> , push via <code>*10</code> ; guard overflow before each push	Overflow guard: check <code>result > Integer.MAX_VALUE/10</code> before pushing	$O(\log n)$	$O(1)$
9	Palindrome Number	Is the integer a palindrome (reads same forwards/backwards)?	Reverse only the second half; compare halves	Half reverse: stop when <code>reversed >= original</code>	$O(\log n)$	$O(1)$
50	Pow(x, n)	Compute <code>x</code> raised to the power <code>n</code>	Fast exponentiation by squaring; handle negative <code>n</code>	Binary exponentiation: <code>result *= x</code> when <code>(n & 1) == 1</code>	$O(\log n)$	$O(1)$
67	Add Binary	Add two binary strings, return the binary sum	Add from right with carry (like #2 but on strings)	Char arithmetic: <code>(a-'0') + (b-'0') + carry</code> , base 2	$O(n)$	$O(n)$
168	Excel Sheet Column Title	Convert column number $\rightarrow$ Excel title ($1 \rightarrow A$, $28 \rightarrow AB$)	Base-26 conversion but 1-indexed ($A = 1$)	1-indexed: <code>n--</code> before each digit extraction	$O(\log n)$	$O(1)$
171	Excel Sheet Column Number	Convert Excel title $\rightarrow$ column number ($AB \rightarrow 28$)	Base-26 parse: <code>result = result*26 + (c-'A'+1)</code>	1-indexed letters: <code>+1</code> so A maps to 1 not 0	$O(L)$	$O(1)$
204	Count Primes	Count primes strictly less than <code>n</code>	Sieve of Eratosthenes; mark multiples non-prime	Start at i*i: smaller multiples already marked	$O(n \log \log n)$	$O(n)$
372	Super Pow	<code>a^b mod 1337</code> where <code>b</code> is a huge array of digits	Process <code>b</code> digit by digit with modular exponentiation	Digit-by-digit: <code>a^b = (a^(b/10))^10 * a^(b%10)</code>	$O(\log n)$	$O(1)$
1201	Ugly Number III	Find <code>n</code> th positive integer divisible by <code>a</code> , <code>b</code> , or <code>c</code>	Binary search on the answer + inclusion-exclusion with LCM	Inclusion-exclusion: count multiples $\leq$ mid via LCMs	$O(\log \min)$	$O(1)$

Canonical Templates

```
// MENTAL MODEL: number = sequence of digits in some base; iterate by repeatedly
//                peeling the last digit (% base) and shifting (/ base).
// WHEN: "reverse digits", "x^n", "base conversion", "count primes", "nth divisible-by".

// — DIGIT EXTRACTION LOOP — peel digits right-to-left
while (n != 0) {
    int digit = n % 10; // last digit
    n /= 10;           // drop last digit
    // ... use digit ...
}

// — FAST POWER (exponentiation by squaring) —
// WHY: x^n needs only O(log n) multiplications by squaring the base
//      and consuming one exponent bit per step.
//   x^13 = x^8 * x^4 * x^1  (13 = 1101 in binary)
double fastPow(double x, long n) {
    double result = 1;
    while (n > 0) {
        if ((n & 1) == 1) result *= x; // bit set → include this power of x
        x *= x;                     // square the base for the next bit
        n >>= 1;                     // consume one exponent bit
    }
    return result;
}

// — BASE CONVERSION — generic base b, 0-indexed
// number → digits: peel with % b, shift with / b, collect, then reverse
StringBuilder builder = new StringBuilder();
while (num > 0) {
    builder.append(num % b);
    num /= b;
}
builder.reverse();
// digits → number: Horner's rule
int value = 0;
for (int digit : digits) value = value * b + digit;

// — SIEVE OF ERATOSTHENES — mark composites up to n
// WHY start at i*i: any multiple k*i with k < i was already marked by k.
boolean[] composite = new boolean[n];
for (int i = 2; i * i < n; i++) {
    if (composite[i]) continue;
    for (int j = i * i; j < n; j += i) // start at i*i, step by i
        composite[j] = true;
}
```

Part 1 — Digit Manipulation

#7 Reverse Integer

Description: Given a signed 32-bit integer `x`, return `x` with its digits reversed. Return 0 if the result overflows.

Intuition: Peel the last digit with `%10` and push it onto a growing `result` with `*10`. The only hazard is overflow, so check capacity *before* the push.

Variation: Overflow guard — before `result = result*10 + digit`, verify `result` is not past `Integer.MAX_VALUE/10` (or below `MIN_VALUE/10`).

```

class Solution {
    public int reverse(int x) {
        int result = 0;
        while (x != 0) {
            int digit = x % 10;
            x /= 10;
            if (result > Integer.MAX_VALUE/10 || result < Integer.MIN_VALUE/10) return 0; // ← VARIATION: overflow guard before pus
            result = result * 10 + digit;
        }
        return result;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#9 Palindrome Number

Description: Return true if integer `x` reads the same forwards and backwards. Negatives are never palindromes.

Intuition: No need to reverse the whole number — build up the reversed second half until it meets or passes the shrinking first half. At the meeting point the two halves should match.

Variation: Half reverse — loop while `x > reversed`; the middle digit (odd-length case) is dropped via `reversed / 10`.

```

class Solution {
    public boolean isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false; // negatives + trailing zero can't be palindrome
        int reversed = 0;
        while (x > reversed) { // ← VARIATION: only reverse half the digits
            reversed = reversed * 10 + x % 10;
            x /= 10;
        }
        return x == reversed || x == reversed / 10; // even length / odd length (drop middle digit)
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

Part 2 — Fast Power

#50 Pow(x, n)

Description: Implement `pow(x, n)`, computing `x` raised to the integer power `n`.

Intuition: Squaring the base doubles the exponent reach, so each bit of `n` decides whether the current power of `x` joins the product — $O(\log n)$ multiplications instead of $O(n)$.

Variation: Handle negative exponent by inverting `x` and negating `N` (use `long` so `-Integer.MIN_VALUE` doesn't overflow); multiply into `result` only when the current exponent bit is set.

```

class Solution {
    public double myPow(double x, int n) {
        long N = n;
        if (N < 0) { x = 1 / x; N = -N; } // ← VARIATION: handle negative exponent (long avoids overflow)
        double result = 1;
        while (N > 0) {
            if ((N & 1) == 1) result *= x; // ← VARIATION: multiply when current bit set
            x *= x; // square base for next bit
            N >>= 1;
        }
        return result;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#372 Super Pow

Description: Compute $a^b \bmod 1337$ where b is given as an array of digits (so b may be astronomically large).

Intuition: Exponents add when powers multiply, and reading b digit by digit means $a^b = (a^{(b/10)})^{10} * a^{(b\%10)}$. Fold left across the digits, applying the modulus at every step to keep numbers small.

Variation: Digit-by-digit modular exponentiation — at each new digit d , raise the running result to the 10th power and multiply by a^d , all mod 1337.

```
class Solution {
    private static final int MOD = 1337;
    public int superPow(int a, int[] b) {
        a %= MOD;
        int result = 1;
        for (int digit : b)
            result = powmod(result, 10) * powmod(a, digit) % MOD; // ← VARIATION: digit-by-digit, (previous)^10 * a^digit
        return result;
    }
    private int powmod(int x, int k) {
        x %= MOD;
        int result = 1;
        for (int i = 0; i < k; i++) result = result * x % MOD; // k <= 10, so plain loop is fine
        return result;
    }
}
```

Time $O(\log n)$ | **Space** $O(1)$

Part 3 — Base Conversion

#168 Excel Sheet Column Title

Description: Given a column number, return its Excel-style title ($1 \rightarrow "A"$, $27 \rightarrow "AA"$, $28 \rightarrow "AB"$).

Intuition: It's base-26, but Excel has no zero digit — columns start at A=1, so the system is 1-indexed (a "bijective base 26"). Decrement before each digit to shift into the standard 0..25 range.

Variation: 1-indexed — do `columnNumber--` before extracting each digit so that $\% 26$ yields 0..25 mapping cleanly onto 'A'..'Z'.

```
class Solution {
    public String convertToTitle(int columnNumber) {
        StringBuilder builder = new StringBuilder();
        while (columnNumber > 0) {
            columnNumber--; // ← VARIATION: 1-indexed → shift to 0-indexed
            builder.append((char)('A' + columnNumber % 26));
            columnNumber /= 26;
        }
        return builder.reverse().toString();
    }
}
```

Time $O(\log n)$ | **Space** $O(1)$

#171 Excel Sheet Column Number

Description: Given an Excel column title (e.g. "AB"), return its column number (28).

Intuition: Horner's rule for base-26, but each letter contributes $c - 'A' + 1$ (A=1, not 0) because the system is 1-indexed.

Variation: 1-indexed letters — $(c - 'A' + 1)$ so 'A' maps to 1 rather than 0.

```

class Solution {
    public int titleToNumber(String columnTitle) {
        int result = 0;
        for (char c : columnTitle.toCharArray())
            result = result * 26 + (c - 'A' + 1);    // ← VARIATION: 1-indexed letters (+1)
        return result;
    }
}

```

Time $O(L)$ L = title length | **Space** $O(1)$

Part 4 — Arithmetic on Strings

#67 Add Binary

Description: Given two binary strings `a` and `b`, return their sum as a binary string.

Intuition: Grade-school addition from the rightmost digit, tracking a carry — identical to adding two linked-list numbers (#2), but the base is 2 and the operands are strings.

Variation: Char arithmetic per column — `(a.charAt(i) - '0') + (b.charAt(j) - '0') + carry`; emit `sum % 2` and carry `sum / 2`.

```

class Solution {
    public String addBinary(String a, String b) {
        StringBuilder builder = new StringBuilder();
        int i = a.length() - 1, j = b.length() - 1, carry = 0;
        while (i >= 0 || j >= 0 || carry != 0) {
            int sum = carry;
            if (i >= 0) sum += a.charAt(i--) - '0';    // ← VARIATION: char → digit
            if (j >= 0) sum += b.charAt(j--) - '0';
            builder.append(sum % 2);                  // ← VARIATION: base-2 digit (sum % 2)
            carry = sum / 2;                          // base-2 carry (sum / 2)
        }
        return builder.reverse().toString();
    }
}

```

Time $O(n)$ n = max length | **Space** $O(n)$

Part 5 — Sieve

#204 Count Primes

Description: Count the number of prime numbers strictly less than `n`.

Intuition: Instead of testing each number for primality, cross out every multiple of each prime once. When you reach an unmarked `i`, it's prime; its multiples below `i*i` were already crossed out by smaller primes, so start marking at `i*i`.

Variation: Start marking at `i*i` — multiples `k*i` with `k < i` were already handled by the smaller factor `k`.

```

class Solution {
    public int countPrimes(int n) {
        if (n < 2) return 0;
        boolean[] composite = new boolean[n];
        int count = 0;
        for (int i = 2; i < n; i++) {
            if (composite[i]) continue;          // already marked → not prime
            count++;
            for (int j = (long)i*i < n ? i*i : n; j < n; j += i) // ← VARIATION: start at i*i (guard i*i overflow)
                composite[j] = true;
        }
        return count;
    }
}

```

Time $O(n \log \log n)$ | **Space** $O(n)$

Part 6 — Binary Search on the Answer

#1201 Ugly Number III

Description: Find the n th positive integer that is divisible by at least one of a , b , or c .

Intuition: The count of valid numbers $\leq x$ is monotonic in x , so binary-search the smallest x whose count reaches n . Counting " $\leq \text{mid}$ divisible by a, b , or c " is a classic inclusion-exclusion over the three sets, subtracting pairwise LCM overlaps and adding back the triple LCM.

Variation: Inclusion-exclusion — $\text{count} = \text{mid}/a + \text{mid}/b + \text{mid}/c - \text{mid}/ab - \text{mid}/bc - \text{mid}/ac + \text{mid}/abc$ using LCMs to avoid double counting.

```

class Solution {
    public int nthUglyNumber(int n, int a, int b, int c) {
        long lo = 1, hi = 2_000_000_000L;
        long ab = lcm(a, b), bc = lcm(b, c), ac = lcm(a, c), abc = lcm(ab, c);
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;
            long count = mid/a + mid/b + mid/c - mid/ab - mid/bc - mid/ac + mid/abc; // ← VARIATION: inclusion-exclusion via LCMs
            if (count < n) {
                lo = mid + 1;          // not enough yet → search higher
            } else {
                hi = mid;              // enough → mid might be the answer
            }
        }
        return (int) lo;
    }

    private long gcd(long x, long y) { return y == 0 ? x : gcd(y, x % y); }
    private long lcm(long x, long y) { return x / gcd(x, y) * y; } // divide first to avoid overflow
}

```

Time $O(\log \min(\text{range}))$ | **Space** $O(1)$

Math Cheat Sheet

```
n % 10          // last decimal digit
n /= 10         // drop last digit
result * 10 + digit // push a digit (build number left-to-right)
(n & 1) == 1     // odd
(n & 1) == 0     // even
n >>= 1        // halve (consume one bit)
x *= x         // square base in fast power
c - '0'        // char → digit value
c - 'A' + 1     // Excel letter → 1-indexed value
(char)('A' + d) // 0..25 → 'A'..'Z'
i * i          // sieve marking start point
x / gcd(x,y) * y // LCM without overflow (divide first)
```

Pattern Chooser

Signal	Technique	Time
"reverse the digits of a number"	Digit pop (%10) / push (*10) loop	O(log n)
"is the number a palindrome?"	Reverse only half, compare halves	O(log n)
"compute x^n / a^b efficiently"	Exponentiation by squaring (binary exp)	O(log n)
"x^huge mod m" / exponent given as array	Digit-by-digit modular fast power	O(log n)
"add two big numbers as strings"	Right-to-left add with carry	O(n)
"convert column number ↔ Excel title"	Base-26 conversion, 1-indexed	O(log n) / O(L)
"convert between number and base-b string"	Horner (parse) / peel-and-reverse (build)	O(L)
"count primes below n"	Sieve of Eratosthenes, mark from i*i	O(n log log n)
"nth number divisible by a/b/c"	Binary search on answer + inclusion-exclusion (LCM)	O(log range)
"check if n is power of 2/4"	Bit trick n & (n-1) == 0 (see bit_manipulation)	O(1)